API Requirements Document

Restful-Booker (Hotel Booking API)

1. Overview

Restful-Booker is a Create, Read, Update, Delete (CRUD) Web API simulating a hotel booking system. It includes token-based authentication and is purpose-built for practicing API testing. This document defines the endpoints, expected behavior, and requirements used as the basis for the accompanying 30-case test case spreadsheet.

Source: https://restful-booker.herokuapp.com

Docs: https://restful-booker.herokuapp.com/apidoc/index.html

2. Base URL

```
https://restful-booker.herokuapp.com
```

3. Resource: Booking

Field	Type	Required	Description
Firstname	string	Yes	Guest's first name
Lastname	string	Yes	Guest's last name
Totalprice	integer	Yes	Total price of the booking
Depositpaid	boolean	Yes	Whether a deposit has been paid
bookingdates.checkin	date (YYYY-MM-DD)	Yes	Check-in date
bookingdates.checkout	date (YYYY-MM-DD)	Yes	Check-out date
Additionalneeds	string	No	Optional extras (e.g. Breakfast)

4. Authentication

4.1 POST /auth

- Purpose: Generate an auth token required for PUT, PATCH, and DELETE requests.

Request Body:

```
{"username": "admin", "password": "password123"}
```

Success Response: 200 OK

```
{"token": " 110c65d5102f246"}
```

- Usage: pass the token in later requests as a cookie header, Cookie: input generated token

5. Endpoints & Functional Requirements

5.1 GET /booking: Get Booking IDs

- Auth: None required
- Optional Query Params: firstname, lastname, checkin, checkout (filters results)
- Success Response: 200 OK — array of { "bookingid": <id>} objects

5.2 GET /booking/{id}: Get Booking

- Auth: None required
- Path Parameter: id (integer, required)
- Success Response: 200 OK full booking object matching the given ID
- Error Case: non-existent ID returns 404 Not Found

5.3 POST /booking: Create Booking

- Auth: None required
- Request Body: full booking object (see Section 3)
- Validation Requirements: all required fields must be present

Success Response: 200 OK

```
{"bookingid":1, "booking": {..same object sent..} }
```

5.4 PUT /booking/{id}: Update Booking

- Auth Required: Yes Cookie: token=<token> header (or Basic Auth)
- Path Parameter: id (integer, required)
- Request Body: full booking object (all fields, since PUT replaces the entire resource)
- Success Response: 200 OK returns updated booking object
- Error Case: missing/invalid token returns 403 Forbidden

5.5 PATCH /booking/{id: Partial Update Booking

- Auth Required: Yes, same as PUT
- Path Parameter: id (integer, required)
- Request Body: only the fields being changed (partial object)
- Success Response: 200 OK, returns updated booking object with only specified fields changed
- Error Case: missing/invalid token returns 403 Forbidden

5.6 DELETE /booking/{id: Delete Booking

- Auth Required: Yes, same as PUT/PATCH
- Path Parameter: id (integer, required)
- Success Response: 201 Created (not 200/204 , a known quirk worth explicitly testing for)
- Error Case: missing/invalid token returns 403 Forbidden

5.7 GET /ping: Health Check

- Auth: None required
- Success Response: 201 Created with body 'Created' (most health checks return 200; this one intentionally does not)

6. Non-Functional Requirements

Requirement	Threshold
Response time (GET requests)	1000ms
Response format	Valid JSON

Requirement	Threshold
Data reset behavior	All data resets to default 10 records every 10 minutes, tests should not assume long-term persistence

7. Status Code Requirements Summary

Status Code	Condition
200 OK	Successful GET, POST (booking creation), PUT, PATCH
201 Created	Successful DELETE; successful /ping health check (both intentional quirks of this API)
403 Forbidden	Missing/invalid auth token on PUT, PATCH, DELETE
404 Not Found	Requested booking ID does not exist

8. Traceability to Test Cases

This requirements document maps to a 30-case test suite, covering positive, negative, boundary, security, and non-functional scenarios across every endpoint.

Requirement Section	Covered By Test Case(s)
Auth: valid credentials	TC_RB_001
Auth: invalid credentials	TC_RB_002
Auth: missing password field	TC_RB_003
Auth: empty request body	TC_RB_004
Create booking: happy path	TC_RB_005
Create booking: missing required field	TC_RB_006
Create booking: invalid data type	TC_RB_007
Create booking: missing nested object	TC_RB_008
Create booking: negative price (boundary)	TC_RB_009
Create booking: optional field omitted	TC_RB_010
Create booking: illogical date range	TC_RB_011
Get all booking IDs	TC_RB_012
Get booking IDs: filter by firstname	TC_RB_013
Get booking IDs: filter by date range	TC_RB_014
Get booking by valid ID	TC_RB_015
Get booking: non-existent ID	TC_RB_016
Get booking: invalid ID format	TC_RB_017
Update booking: valid token	TC_RB_018
Update booking: without token	TC_RB_019
Update booking: invalid/expired token	TC_RB_020
Update booking: missing required field	TC_RB_021
Partial update: single field	TC_RB_022
Partial update: multiple fields	TC_RB_023
Partial update: without token	TC_RB_024
Delete booking: with valid token	TC_RB_025
Delete booking: without token	TC_RB_026
Delete booking: idempotency (delete again)	TC_RB_027
Health check ping	TC_RB_028
Response time requirement	TC_RB_029
Content-Type header requirement	TC_RB_030